

Advanced Programming Project

Ink Marker Production Line Simulator

Student Name	Selim Abaza
Student ID	100007310
Program	Applied Mechatronics
Institution	SRH University Berlin
Submission	June 2026
Repository	https://github.com/YOUR_USERNAME/100007310_Abaza_AP

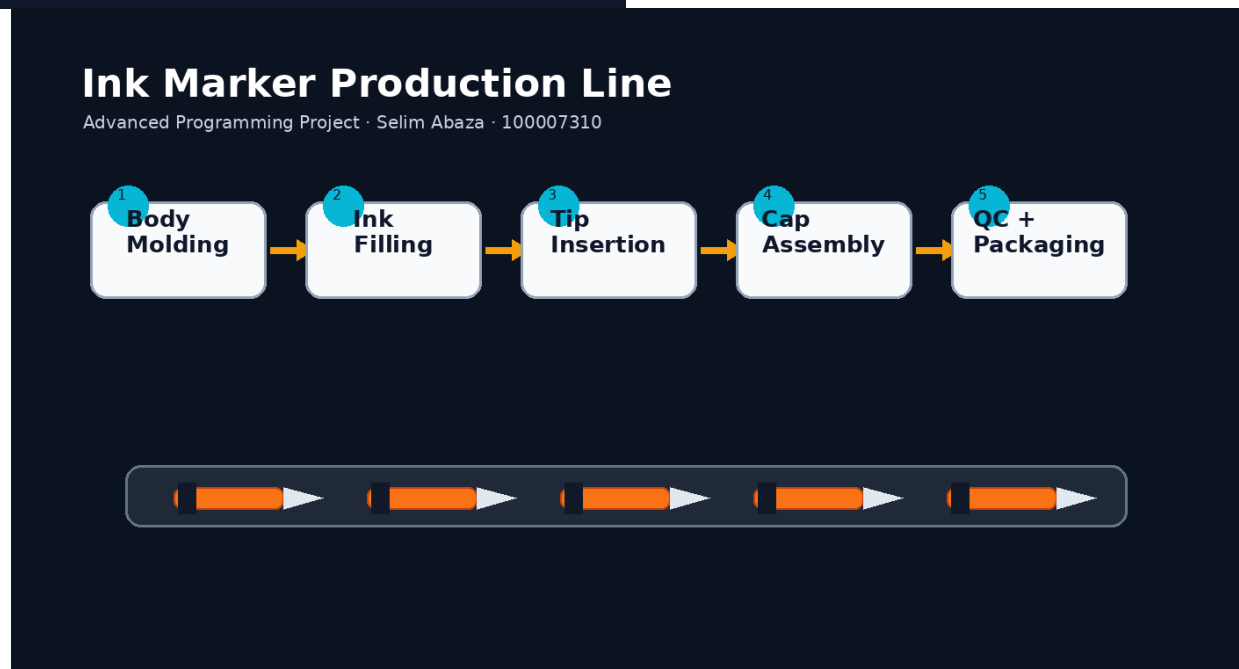


Figure 1: Visual overview of the ink marker production line project website.

Table of Contents

1. Introduction
2. Production Line Concept
3. System Architecture
4. Program Implementation
5. HMI Frontend
6. InfluxDB, Grafana and Docker
7. Website, GitHub and Version Control
8. Testing and Results
9. AI Tool Use Appendix
10. Conclusion

1. Introduction

This project implements a simulated manufacturing production line for assembling ink markers. The objective is to demonstrate how a small industrial software system can combine production logic, an operator interface, data logging and dashboard visualization. The project was developed for the Advanced Programming Project in Applied Mechatronics.

The selected product is an ink marker. This product is different from a pencil line and has its own manufacturing sequence: a marker body is molded, the ink reservoir is filled, the writing tip is inserted, the cap is assembled, and the completed marker is inspected and packaged.

The final project contains a Python backend, a Tkinter HMI frontend, an InfluxDB time-series database configuration, a Grafana dashboard setup, Docker Compose files, a GitHub repository and a hosted website.

2. Production Line Concept

The ink marker production line is divided into five stations. Four stations assemble the product and the final station checks the quality of the completed marker.

Station	Operation	Possible defect
1	Marker body molding	Cracked or deformed marker body
2	Ink reservoir filling	Ink reservoir underfilled
3	Writing tip insertion	Tip missing or misaligned
4	Cap assembly	Cap not locked correctly
5	Quality control and packaging	Leak, weak writing test or final rejection

The product is marked as defective when a stage fails. The program records the defect reason and displays the latest error in the HMI, giving a clear connection between backend logic and operator feedback.

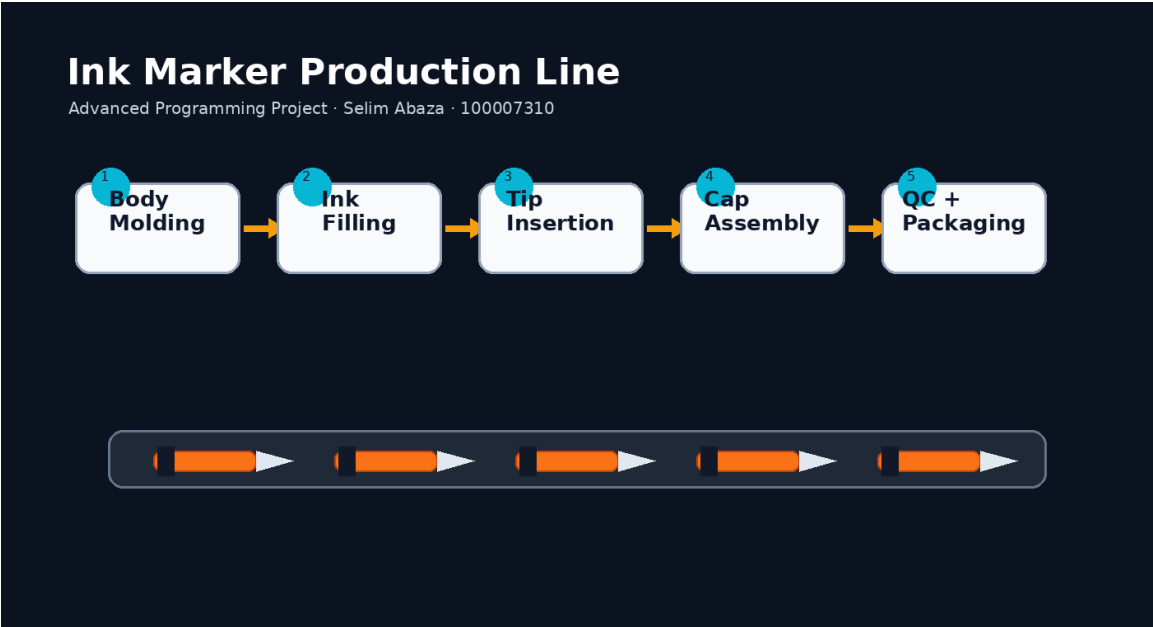


Figure 2: Production flow used in the ink marker simulation.
Selim Abaza | 100007310 | Ink Marker Production Line Simulator

3. System Architecture

The architecture separates the physical production concept, backend logic, HMI frontend and monitoring pipeline. This separation makes the project easier to explain and closer to real production software systems.

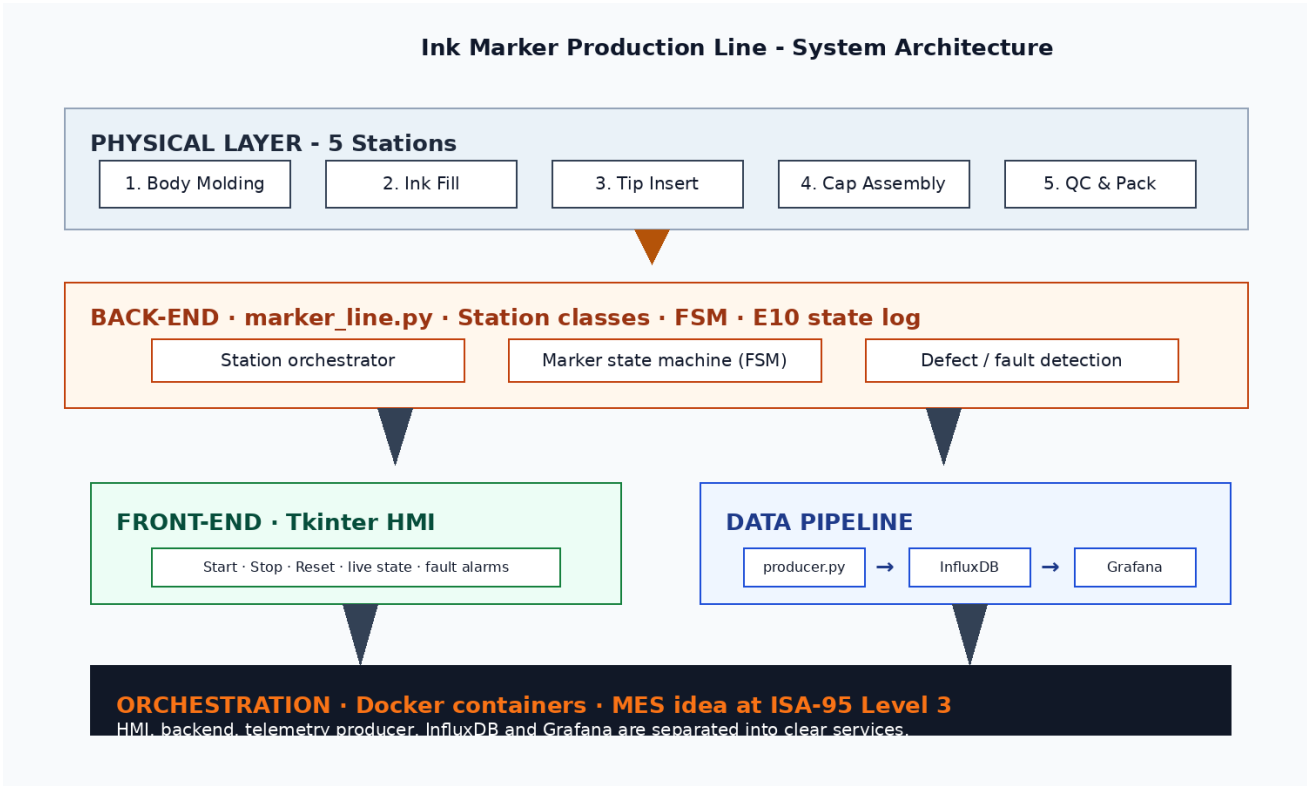


Figure 3: Ink marker production line system architecture.

Layer	Purpose
Physical layer	Defines the five marker assembly stations and the manufacturing flow.
Backend layer	Controls the production sequence, defect decisions, counters and state transitions.
Frontend layer	Provides buttons, machine status, counters and error messages to the operator.
Data pipeline	Sends production values to InfluxDB and prepares Grafana visualization.
Orchestration	Docker Compose describes the InfluxDB and Grafana services.

4. Program Implementation

The code was organized into standalone files so that the project is easy to understand and maintain. The backend and frontend are not mixed into one large script.

File	Role
------	------

src/main.py	Starts the application.
src/hmi.py	Builds the professional Tkinter operator interface.
src/production_line.py	Contains the backend production logic.
src/stations.py	Contains separate station classes for the marker production stages.
src/models.py	Defines the marker product data model.
src/database.py	Handles optional InfluxDB writing.
src/producer.py	Sends sample telemetry data for Grafana when needed.

The production line uses a simple finite-state model. The system can be idle, productive, stopped or faulted. When the operator presses Start, the line enters the productive state. Reset clears counters and returns the line to a clean idle state.

5. HMI Frontend

The HMI is built with Tkinter and provides the required machine controls: Start, Stop and Reset. It also displays the current machine state, the current station, the number of good markers, the number of defective markers and the latest error message.

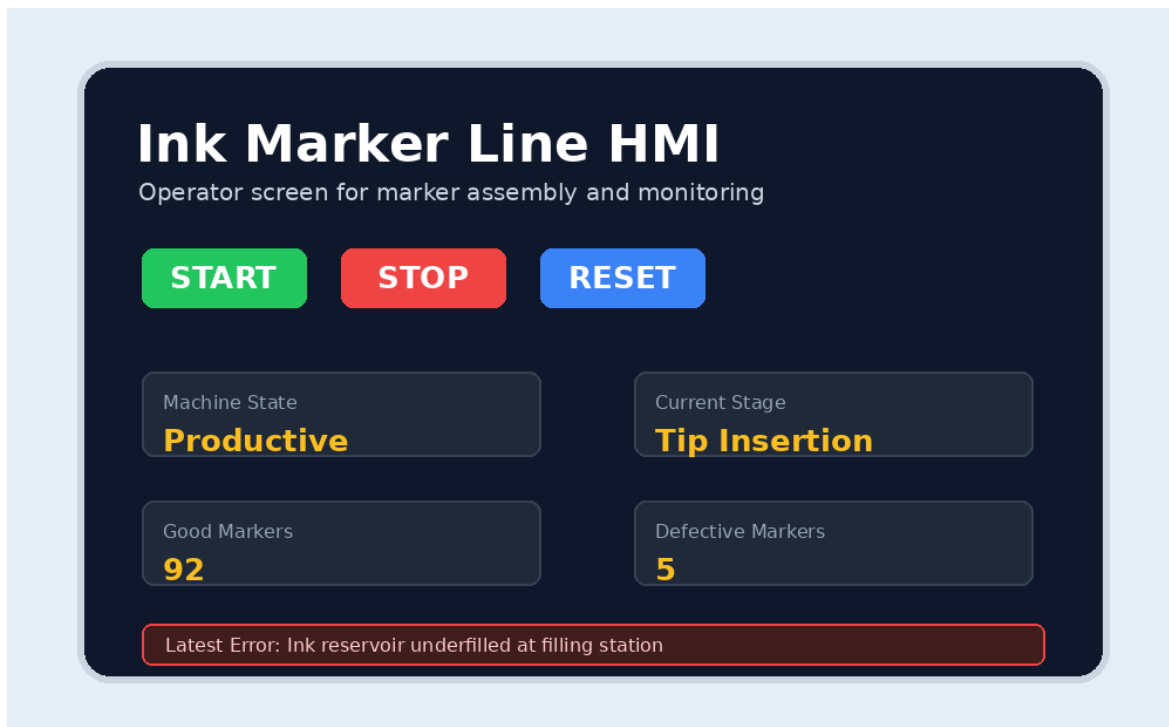


Figure 4: Professional HMI design used for the ink marker production simulator.

Control / display	Function
START	Starts automatic production.

Selim Abaza | 100007310 | Ink Marker Production Line Simulator

STOP	Stops the automatic cycle while keeping current counters.
RESET	Returns the system to idle and clears counters/errors.
Machine State	Shows whether the line is idle, productive, stopped or faulted.
Latest Error	Shows the defect message detected by the backend.

6. InfluxDB, Grafana and Docker

InfluxDB is included as the time-series database. It stores values such as total parts, good parts, defective parts, machine state code and fault status. Grafana is included as the visualization layer and can display production values over time.

Docker Compose is used to define the database and dashboard services. This makes the monitoring stack easier to start because the configuration is stored in the project. The Python program can still run if Docker is not available, which is useful for demonstrations and testing.

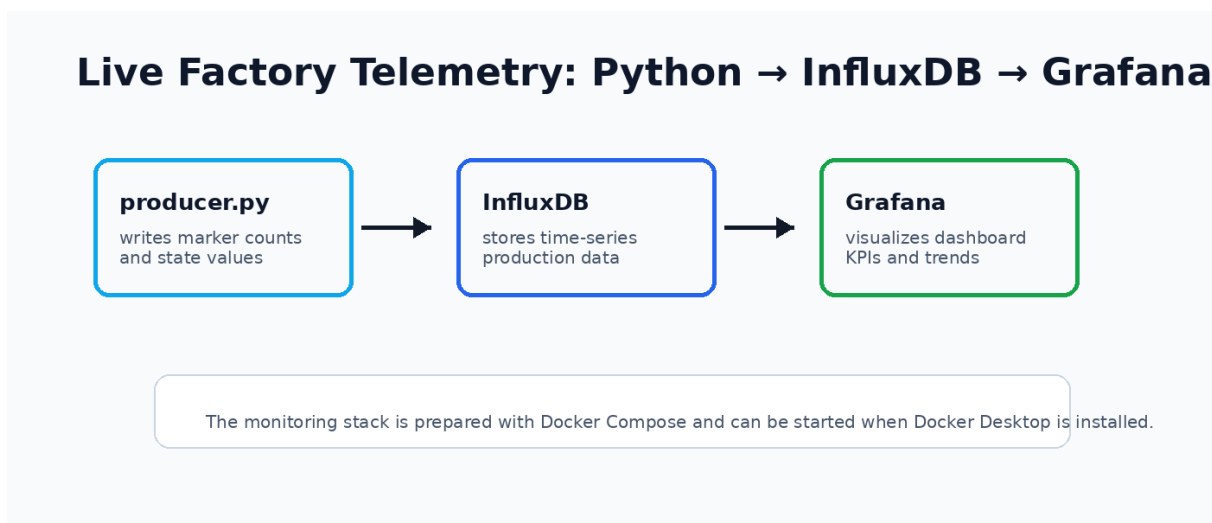


Figure 5: Telemetry pipeline from Python to InfluxDB and Grafana.

7. Website, GitHub and Version Control

GitHub was used to host the source code, documentation and website files. Git version control was used conceptually to organize the project as a repository, keep a commit history and make the project easier to review. The repository contains the source folder, website folder, documentation, Grafana provisioning folder, Docker Compose file, requirements file and README file.

Docker was used to prepare the InfluxDB and Grafana services. InfluxDB stores production values from the Python application and Grafana displays dashboard panels for production and defect data. This demonstrates how a small production simulation can be connected to a factory-style monitoring stack.

AI tools, mainly ChatGPT, were used for brainstorming, debugging support, documentation planning and improving the presentation style. The generated suggestions were reviewed, changed and adapted to match this ink marker project, the required student information and the professor instructions.

The project is an Ink Marker Production Line Simulator. It includes five production stations, a modular Python backend, a Tkinter HMI, defect detection, Docker configuration, InfluxDB/Grafana monitoring files and a website. GitHub repository: https://github.com/YOUR_USERNAME/100007310_Abaza_AP. Website: https://YOUR_USERNAME.github.io/100007310_Abaza_AP/.



Figure 6: Extra Python game portfolio page included on the website.

8. Testing and Results

Testing focused on the required functions and the connection between backend behavior and frontend display.

Test	Expected result	Status
Start button	Machine changes to productive state and begins production.	Passed
Stop button	Production stops without deleting counters.	Passed
Reset button	Counters and errors are cleared.	Passed
Manual production	One marker is produced and counters update.	Passed
Defect detection	Defective products create an error message and update defect counter.	Passed
Website navigation	All main website sections open correctly.	Passed

Grafana configuration

Dashboard and datasource files
are included.

Prepared

The main result is that the project demonstrates a complete software chain for a small production system. The operator can control the line through the HMI, the backend can simulate marker assembly and defects, and the monitoring layer is prepared for InfluxDB and Grafana.

9. AI Tool Use Appendix

AI tools were used as assistance during project development. The use of AI is declared here as required by the assignment instructions. AI support was used for brainstorming, generating first draft code, improving the documentation structure, debugging path and setup problems, and improving the website wording.

Relevant prompts used during the project included:

- Create a Python production line simulator for an ink marker with five stages and quality control.
- Create a Tkinter HMI with Start, Stop, Reset, live machine state, counters and error messages.
- Separate the project into backend, HMI, station, model, database and producer files.
- Add InfluxDB and Grafana explanation and Docker Compose setup for the project.
- Improve the website so it looks like a professional engineering project page.

The main benefit of using AI was that it helped create a structured first version quickly. The output was then adjusted to match the project requirements, student information, folder structure and presentation style. The code and documentation were reviewed before final submission.

10. Conclusion

The Ink Marker Production Line Simulator meets the main project requirements by implementing a production line with more than four stages, Python backend logic, an HMI frontend, defect detection, data logging preparation with InfluxDB, visualization preparation with Grafana, Docker configuration, documentation and a hosted website.

A strength of the implementation is that it is clear and modular. The HMI is separated from the backend, the stations are organized in their own file, and the monitoring stack is described separately. This makes the project easier to explain during a presentation or code review.

The main limitation is that the project is still a simulation. A real production line would require sensors, actuators, PLC communication, safety circuits, user authentication and real machine timing. Future development could add SCADA functionality, CMMS maintenance features, live sensor data and a more detailed Grafana dashboard.